CS5013 Final Project Report

IITM Explainable Slotbook

Nishanth S, EE23B049 Sparsh Gupta, EE23B117

Team Name: TT generator
Repository: `https://github.com/lazarsalt1441/CS5013-PROJECT`
Submission commit: `final-submission` = `33789f4222c08f9089d44b3b32230e458e4941c2` (date: `2026-11-08`).
Docker image: `https://drive.google.com/drive/folders/1QE943nGJ2SliXU7PvFqpfCq07O2-aM_q`
Tagline: *It builds an IITM department's weekly lecture-slot timetable without breaking any hard rule, and explains each course's slot.*

AI-authorship clause. *We used AI tools to help draft parts of this report. The claims, numbers, commit references, stakeholder statements, and the Docker image are our own; the prose shaping may have been assisted. We can defend every sentence of this report, and every line of code in the submission commit, in the viva.*

1 Executive summary

IITM departments assign lecture slots by hand-editing last year's slotbook, so nobody can say why a course got its slot.

We shipped an engine that solves the timetable with UniTime's cpsolver (CP-SAT as an exact reference) and gives a reason for every assignment and every rejected slot. Students and professors submit preferences on a web portal, which warns students about missing prerequisites from their grade card; the admin clicks "Generate slotbook" and the explained timetable is published. The portal and prerequisite warnings were added after the proposal.

Stakeholder verdict: Accepted (final demo on Friday, 6 November 2026).

2 Course concepts used in the project

Java paths are under `src/main/java/in/ac/iitm/slotbook/`.

- **Module 2 – Prompt engineering (CoSTAR, negative prompts).** All 20 AI tasks were logged before coding, as the request, a CoSTAR restatement and negative constraints added at review. `prompts/PROMPT_LOG.md`; `ebb9c28` (P015, logged before `7591cca`).
- **Module 3 – Git, Conventional Commits and CI.** Branches and pull requests; GitHub Actions runs both suites against a Postgres service, plus CLI runs on both solvers. `.github/workflows/ci.yml`; `8f2c000` (PR #1).
- **Module 4 – Structured debugging.** 14 faults recorded from reproduction to regression test, e.g. a race that returned 500s under 25 concurrent writers. `docs_v3/DEBUGGING.md`; `2607a9d`.
- **Module 4 – Static analyser first, then the smallest patch.** SpotBugs reported 10 findings; 3 were a real bug (day names broke under a Turkish locale). Now 0, and any new one fails CI. `spotbugs-exclude.xml`; `374d4e7`.
- **Module 5 – Independent oracles and stress testing.** A separate checker, penalty calculator and brute-force solver judge both solvers; fuzzing and 300 concurrent requests found 3 faults. `src/test/.../testing/ConstraintChecker.java`; `efdf10d`, `2607a9d`.
- **Module 5 – Property-based and mutation testing.** jqwik found on its first run that meetings that only touch were counted as clashes; PIT raised the mutation score from 91.0% to 98.7% (2 equivalent mutants left), now gated at 97%. `src/test/.../model/SlotPropertiesTest.java`; `82cc7f1`, `fc462ab`.
- **Module 5 – Documentation as code.** A man page linted by CI, a manual, and a test plan listing every test. `man/slotbook.1`, `docs_v3/TEST_PLAN.md`; `9ee919b`.

- **Module 7 – Licences and provenance.** Licences of all 16 runtime dependencies are generated in CI and shipped in the image; LGPL cpsolver stays a replaceable jar. `THIRD_PARTY_NOTICES.md`; `406b360`.
- **Module 9 – Supply chain and secrets.** Lockfile with `npm ci`, `npm audit` gate (0 vulnerabilities), `DATABASE_URL` only from the environment, a non-root image scanned for secrets. `.github/workflows/quality.yml`; `95fb07f`, `f47ae1c`.

3 AI-usage summary

Tools used. Claude Code about 50% (engine, portal, tests, docs); ChatGPT about 50% (design, Vercel, Neon and Render setup).

One substantive help. Claude Code replaced the Google Sheets backend with a Postgres store, numbered migrations and database-backed tests (`portal/api/_store.js`, `7591cca`); it shipped near as-is once 4 integration tests passed on a real Postgres 16.

One substantive failure caught in review. The AI-written admin-dashboard check counted U and W grades as completed, contradicting our rule (spec drift); a golden file shared with the Java CLI caught it before merge (`portal/api/_prerequisites.js`, `f528505`).

Net-effect sentence. Net help, with a condition: easy to build with but even easier to get lost, so we built features slowly and in stages, never in one shot. Of the 14 faults in AI-assisted code recorded in `docs_v3/DEBUGGING.md`, every one was caught by a test or a tool, not by reading.

4 Deployment and stakeholder handoff

Build and run.

```
docker load -i slotbook-33789f4.tar.gz
docker run --rm slotbook:33789f4        # solves data/feasible, prints the slotbook
mkdir -p out && docker run --rm -u "$(id -u):$(id -g)" -v "$PWD/out:/out" \
    slotbook:33789f4 --input data/demo-showcase --output /out   # CSVs land in ./out
```

The image holds the engine, CLI and solver web service (`docker run ... serve`); the portal runs on Vercel.

Where the artefact lives after the course. Code on GitHub; portal at `https://cs5013-project-portal.vercel.app`; solver service at `https://slotbook-solver.onrender.com`. Both run on a team member's accounts, to be handed over to the institute administration.

Stakeholder quote (one sentence, verbatim).

> *"The project works as per the expectations given. To integrate it with IITM's data, the admin would need to make some modifications to get data from their databases, but this is good enough from your side."*
> *– Meghana Nasre, in charge of the slotbook, CS Department, 6 November 2026.*

5 Known limitations

- **Optimality.** UniTime is a heuristic: on 3 of 13 generated scenarios it finished 1–8 points above the CP-SAT optimum.
- **No identity on the portal.** Anyone can submit under any student ID, and grade cards are self-uploaded and unverified. It is not fit for real student data until IITM login exists.
- **Fixed rooms.** Capacity is not checked: a course larger than its room, or one needing a room chosen, is not handled.

6 Individual contributions *(omit if solo)*

- **Nishanth S:** built v1 (CP-SAT model, explanations, CLI, JavaFX view, CI) and v3 (Postgres, prerequisite warnings, solver service, Docker); tested with oracles, stress, property and mutation tests.
- **Sparsh Gupta:** built v2 (UniTime solver, real IITM time grid, Sheets-backed portal) and the mid-evaluation showcase; tested UniTime against CP-SAT on every scenario.
- **Both:** stakeholder liaison with Meghana Nasre, demos and requirement changes.